Labsheet 10 : BTCS301P-AS010

Computer Vision Lab

Khushi Choudhary CU240251564

```
# Step 2: Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

```
# Step 1 & 3: Load and Preprocess Dataset
(X_train_raw, y_train_raw), (X_test, y_test) = mnist.load_data()
X_train, X_val = X_train_raw[:50000], X_train_raw[50000:]
y_train, y_val = y_train_raw[:50000], y_train_raw[50000:]

X_train = X_train.reshape((-1, 28, 28, 1))
X_val = X_val.reshape((-1, 28, 28, 1))
X_test = X_test.reshape((-1, 28, 28, 1))

# Normalize pixel values from [0, 255] to [0.0, 1.0]
X_train = X_train.astype("float32") / 255.0
X_val = X_val.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
print(f"Training shape: {X_train.shape}, Labels: {y_train.shape}")
print(f"Validation shape: {X_val.shape}, Labels: {y_val.shape}")
print(f"Test shape: {X_test.shape}, Labels: {y_test.shape}")
```

```
Training shape: (50000, 28, 28, 1), Labels: (50000,)
Validation shape: (10000, 28, 28, 1), Labels: (10000,)
Test shape: (10000, 28, 28, 1), Labels: (10000,)
```

```
# Step 4: Design CNN Architecture
model = models.Sequential([
    # First Convolutional Block
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    # Second Convolutional Block
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    # Fully Connected Classifier
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),  # Regularization to reduce overfitting
    layers.Dense(10, activation='softmax')  # 10 output classes
])
model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential mo
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

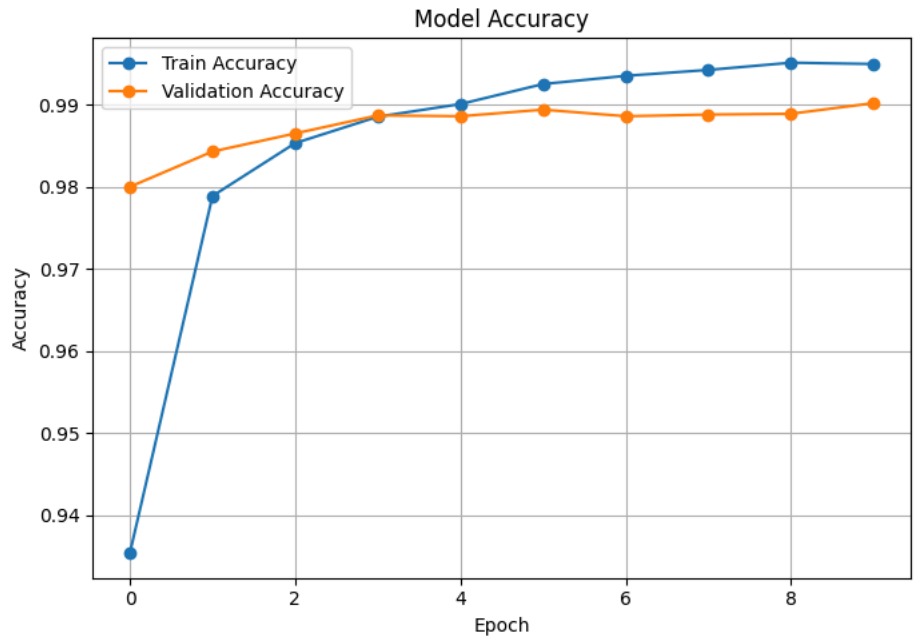
```
# Step 5 & 6: Compile and Train Model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
EPOCHS = 10
BATCH_SIZE = 64
history = model.fit(
    X_train, y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(X_val, y_val),
    verbose=1
)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

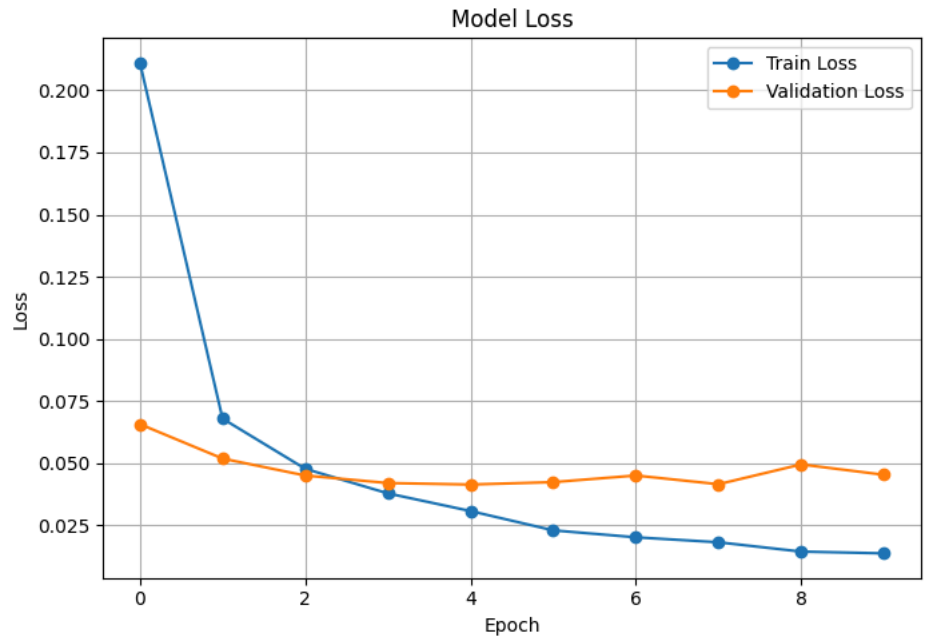
# Accuracy plot
ax1.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
ax1.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='o')
ax1.set_title('Model Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax1.grid(True)

# Loss plot
ax2.plot(history.history['loss'], label='Train Loss', marker='o')
ax2.plot(history.history['val_loss'], label='Validation Loss', marker='o')
ax2.set_title('Model Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
ax2.grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 49s 60ms/step - accuracy: 0.9354 - loss: 0.2111 - val_accuracy: 0.9800 - val_loss: 0.0657
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 45s 57ms/step - accuracy: 0.9789 - loss: 0.0679 - val_accuracy: 0.9843 - val_loss: 0.0519
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9853 - loss: 0.0478 - val_accuracy: 0.9865 - val_loss: 0.0451
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.9885 - loss: 0.0379 - val_accuracy: 0.9887 - val_loss: 0.0420
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 45s 58ms/step - accuracy: 0.9901 - loss: 0.0308 - val_accuracy: 0.9886 - val_loss: 0.0414
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 81s 56ms/step - accuracy: 0.9925 - loss: 0.0230 - val_accuracy: 0.9894 - val_loss: 0.0424
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.9935 - loss: 0.0202 - val_accuracy: 0.9886 - val_loss: 0.0451
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 44s 56ms/step - accuracy: 0.9942 - loss: 0.0182 - val_accuracy: 0.9888 - val_loss: 0.0416
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 45s 58ms/step - accuracy: 0.9951 - loss: 0.0145 - val_accuracy: 0.9889 - val_loss: 0.0495
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 80s 56ms/step - accuracy: 0.9950 - loss: 0.0138 - val_accuracy: 0.9902 - val_loss: 0.0454
```





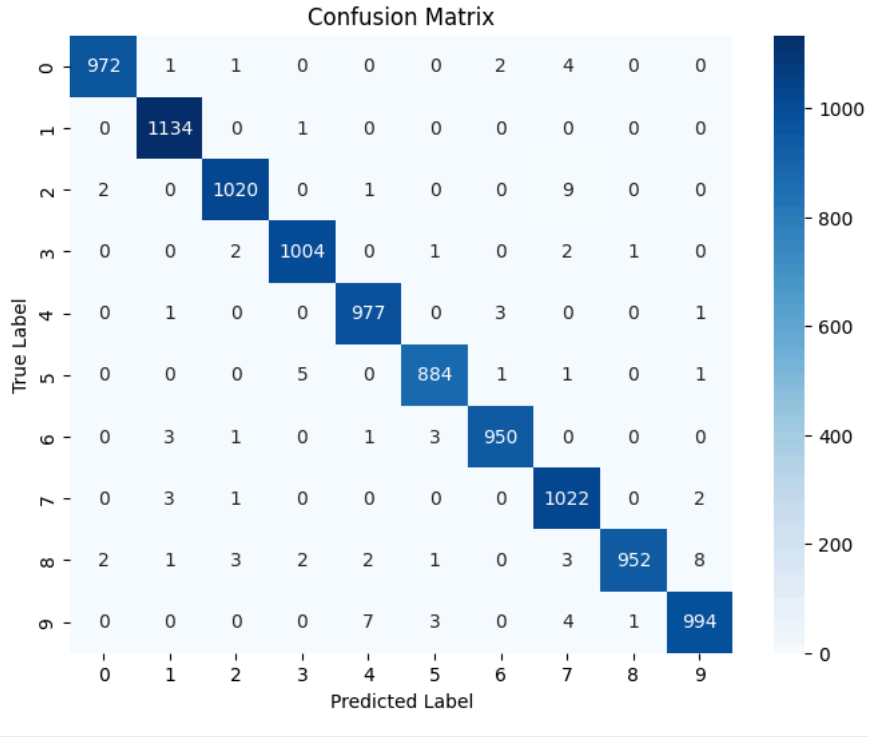

```
# Step 7: Evaluate on Test Set
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
print(f"Test Loss: {test_loss:.4f}\n")
# Get class predictions
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
# Precision, Recall, F1-Score
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=4))
# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=range(10), yticklabels=range(10))
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Test Accuracy: 99.09%
Test Loss: 0.0313

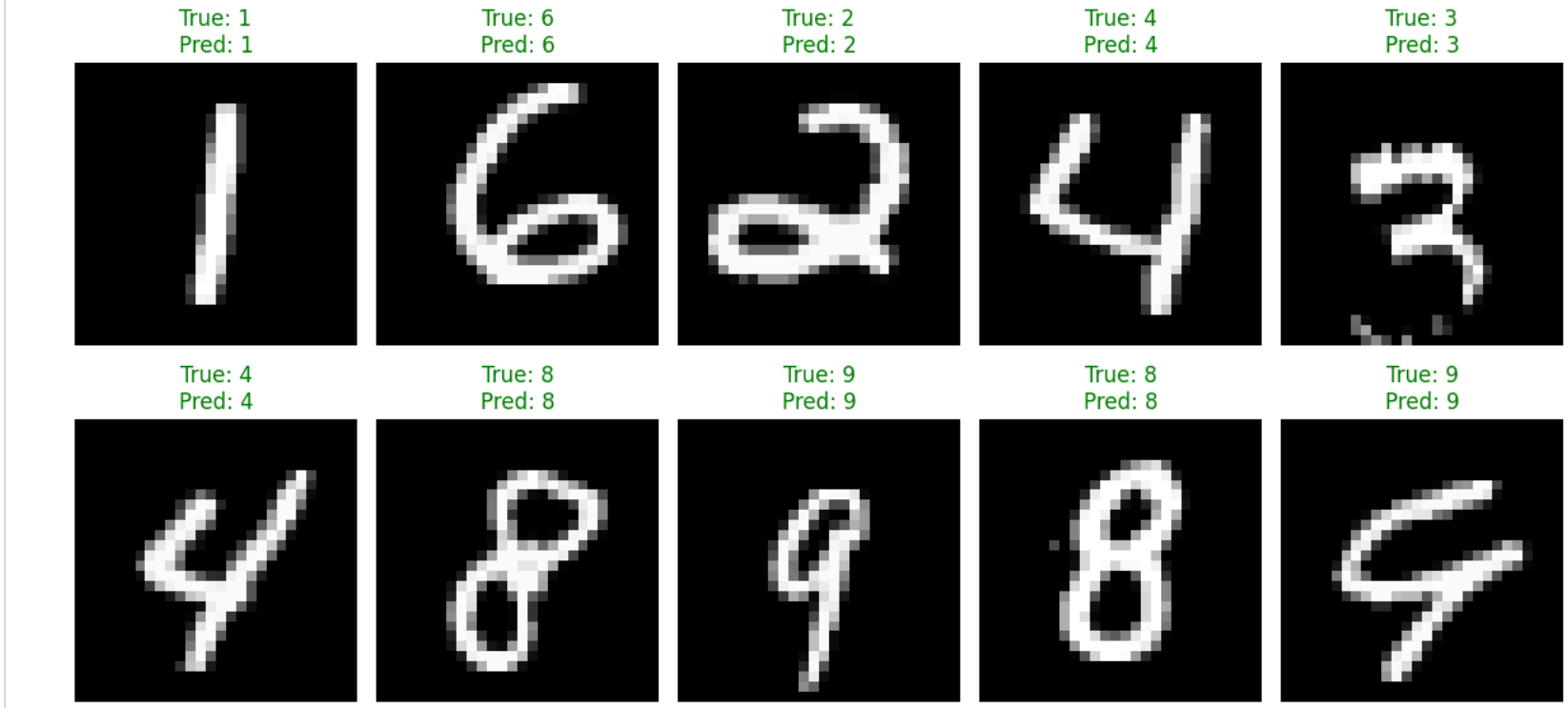
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step
Classification Report:
              precision    recall  f1-score   support

           0     0.9959    0.9918    0.9939       980
           1     0.9921    0.9991    0.9956      1135
           2     0.9922    0.9884    0.9903      1032
           3     0.9921    0.9941    0.9931      1010
           4     0.9889    0.9949    0.9919       982
           5     0.9910    0.9910    0.9910       892
           6     0.9937    0.9916    0.9927       958
           7     0.9780    0.9942    0.9860      1028
           8     0.9979    0.9774    0.9876       974
           9     0.9881    0.9851    0.9866      1009

    accuracy                         0.9909     10000
   macro avg     0.9910    0.9908    0.9909     10000
weighted avg     0.9909    0.9909    0.9909     10000
```



```
# Step 8: Predict on Unseen Images
plt.figure(figsize=(12, 6))
for i in range(10):
    idx = np.random.randint(0, len(X_test))
    img = X_test[idx]
    true_label = y_test[idx]
    pred_label = y_pred[idx]
    plt.subplot(2, 5, i + 1)
    plt.imshow(img.squeeze(), cmap='gray')
    color = 'green' if true_label == pred_label else 'red'
    plt.title(f"True: {true_label}\nPred: {pred_label}", color=color)
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Step 9-10: Factors Affecting Performance:

Hyperparameters (learning rate, batch size), model capacity, depth, and dataset balance strongly influence final accuracy.

Misclassification Causes: Common errors occur on ambiguous digits (e.g., distinguishing an elongated 4 from a 9, or a poorly drawn 7 from a 1).

Real-World Applications: Form automated processing, OCR systems, cheque verification, and licence plate recognition.